

LB1: Unit-Testing

1 Ausgangslage

Unser Unternehmen wird eine Open-Source-Softwarelösung verwenden, die für unsere geschäftlichen Anforderungen gut geeignet ist. Damit sicher ist, dass alles gut funktioniert, werden zu einzelne Prozeduren Unit-Tests verlangt.

Obwohl die Open-Source-Softwarelösung die notwendigen Grundfunktionen bietet, haben wir spezifische Anforderungen identifiziert, die über die vorhandenen Funktionen hinausgehen. Um diesen Anforderungen gerecht zu werden, beabsichtigen wir, zusätzliche Funktionen mittels Test-Driven-Development selbst zu entwickeln.

2 Aufgabenstellung

2.1 Produkt

Sie schreiben Unit-Test und Programm-Code. Diese stellen Sie der Lehrperson vor am dritten Unterrichtstag.

2.2 Sozialform

Die Bearbeitung dieses Auftrags erfolgt in Partnerarbeit.

2.3 Test-Software

Sie wählen eine Software aus, die Sie für LB1 und LB2 als Basis für Ihre Testumgebung benötigen.

Bedingungen

Die zu testende Software muss folgende Bedingungen erfüllen:

- Die Software muss über eine implementierte (Geschäfts-)Logik verfügen, die spezifische (Geschäfts-)Prozesse abbildet.
- Die Technologie muss ein Unit-Test-Framework haben,
- UI ist nicht zwingend.

E-Mail

Wenn Sie Ihre Auswahl getroffen haben, schreiben Sie eine E-Mail an die Lehrperson und CC an Ihren Teampartner mit Betreff "M450 <Klasse> Software" und folgenden Inhalt:

- Zusammensetzung des Teams
- Name und Quelle des Programms
- Kurze Beschreibung, was das Programm macht
- Eingesetzte Technologien (Programmiersprache, DB, Entwicklungsumgebung, ...)
- Liste der wichtigsten Entitäten (mindestens 3).

2.4 White-Box Testing:

Sie erstellen einige Unit-Tests für den vorhandenen Programmcode.

2.4.1 Vier Test-Kategorien

Die ersten Unit-Tests sollen eine oder mehr Code-Units komplett testen mit vier verschiedene Test-Kategorien aus der untenstehenden Tabelle.

Test-Kategorie	Beschreibung / Fokus
Diskrete Grenzwerte	Ganzzahlen an exakten Bereichsgrenzen (z. B. $n-1$, n , $n+1$)
Kontinuierliche Grenzwerte	Gleitkommazahlen mit Toleranzbereichen (z. B. 3.14 ± 0.001)
Format & Muster (Regex)	String-Validierung über Muster (E-Mail, PLZ, IBAN, Passwort)
Kombinatorische Logik	Verknüpfte Bedingungen (AND / OR / Entscheidungstabellen)
Fehlerbehandlung & Edge Cases	Gezieltes Abfangen von <code>null</code> , unzulässigen Werten oder Exceptions
Zustandsbasiertes Verhalten	Verhalten der Methode abhängig vom Objekt-Zustand (State Machine)
Asynchronität / Promises	Testen von <code>async/await</code> , Promises oder Timeout-Verhalten
Sammlungen & Listen	Testen von Arrays/Listen (Filterung, Sortierung, leere Listen)

Pro Test-Kategorie sollen alle möglichen Fälle getestet werden. Denken Sie hierbei unter anderem an Positiv- und Negativtests und Boundary-Value-Analysis.

Wenn Sie andere Test-Kategorien einsetzen möchten, dann besprechen Sie dies vorher mit der Lehrperson.

Hinweis: Sie können die 4 Kategorien an einer einzigen komplexen Methode oder verteilt über mehrere Methoden Ihres Programms testen.

2.4.2 Freier Wahl

Der zweite Unit-Test zeigt Ihre Kompetenz in fortgeschrittenen Testing-Techniken. Mögliche Themen sind:

- **Time-Freezing / Clock Mocking:** Simulation von Zeitabläufen (z. B. Ablauf von Fristen, Schaltjahre, Mitternacht) mittels Zeit-Bibliothek.
- **Automatisierte Code-Mutation:** Überprüfung der Testqualität durch automatisches Einbauen von Code-Fehlern (Mutation Testing Tools).
- **Erweitertes Mocking:** Komplexe Behavior-Checks (z. B. Prüfen von Aufrufreihenfolgen oder Exception-Simulationen bei Schnittstellen).

Multiplikationsfaktor für Test 2:

- **Faktor 1:** Sie setzen lediglich ein einfaches Mocking ein (Standard-Fall).
- **Faktor 2:** Sie setzen Time-Freezing oder erweitertes Mocking fachlich korrekt um.

- **Faktor 3:** Sie setzen automatisierte Code-Mutation (Mutation Testing) ein oder kombinieren mehrere Spezialtechniken sinnvoll.

Wenn Sie andere Testing-Techniken einsetzen möchten, dann besprechen Sie dies vorher mit der Lehrperson.

2.5 Test-Driven-Development

- 1) Sie erstellen erst einen Unit-Test für nicht-existierenden Programmcode. Dieser Test muss einen spezifischen Fehlerfall oder ein fehlendes Feature abdecken.
- 2) Nachher schreiben Sie (oder eine KI) den Programmcode für den Unit-Test. Fügen Sie eventuell absichtlich einen kleinen Fehler in den Code ein, welche durch den Unit-Test gefunden wird.
- 3) Dokumentieren Sie den Fehler und die Korrekturmassnahme klar und präzise in der Vorlage M450_LB1_Unittest_Fehler.xlsx. Der Fehler muss nicht gelöst werden: Status darf auf "Offen" bleiben.

3 Bewertung

3.1 Abgabezeitpunkt

Für die Bewertung zeigen Sie der Lehrkraft Ihre Ergebnisse (Programmcode). Sie können dies tun, sobald Sie bereit sind, einen oder mehrere Teile zu zeigen, spätestens jedoch am Ende der Lektion in der dritten Quartalswoche.

3.2 Kriterien und Erklärung

Die Bewertung der Unit-Tests basiert auf einer Kombination aus Code-Komplexität, fachlicher Relevanz der Test-Idee, der Fähigkeit des Tests, neue Fehler aufzudecken, und dem Grad der Automatisierung.

Siehe auch das separate **Bewertungsraster**.

A) Kriterium: Fachliche Relevanz und Korrektheit

Die Punkte für den Unit-Test 2 werden mit einem Faktor von 0 bis 3 **multipliziert**, basierend auf der Qualität der Idee und Umsetzung:

- **Idee ist sinnvoll:** Die Idee des Tests muss eine relevante und kritische Funktion der Anwendung abdecken und einen Mehrwert bieten (z.B. ein komplexes Business-Regelwerk prüfen).
- **Korrekte Umsetzung:** Der Test muss fachlich korrekt implementiert sein und das erwartete Verhalten eindeutig verifizieren.

Wichtig: Die Sinnhaftigkeit der Idee wird durch die Lehrperson festgelegt. Eine **vorherige Absprache** mit der Lehrperson wird dringend empfohlen, um die maximale Punktzahl zu gewährleisten.

Wichtig: Wenn Sie nur Mocking einsetzen, bekommen Sie eine Multiplikationsfaktor=1. Schauen Sie, dass Sie noch etwas mehr machen, um den Faktor auf 3 zu erhöhen.

B) Identifikation neuer Fehler (Tatsächliche Wirksamkeit)

Die Unit-Tests müssen in der Lage sein, tatsächliche (neue) Fehler und Probleme aufzuzeigen.

Die Lehrperson wird Änderungen am Code vornehmen, um die Robustheit und Wirksamkeit Ihrer Unit-Tests zu prüfen. Die Fähigkeit des Unit-Tests, diese vom LP verursachten Probleme zu erkennen, fließt in die Bewertung ein.

C) Automatisierungsgrad des Tests

Der Automatisierungsgrad des Tests ergibt folgende Punkte:

- Manuell: 0 Pkt.
- Bat: 1 Pkt.
- Sched: 2 Pkt.
- Nach «push»: 3 Pkt.